CSC 113 : End Exam

Java Programming (Wednesday Version 1)

Name: .....

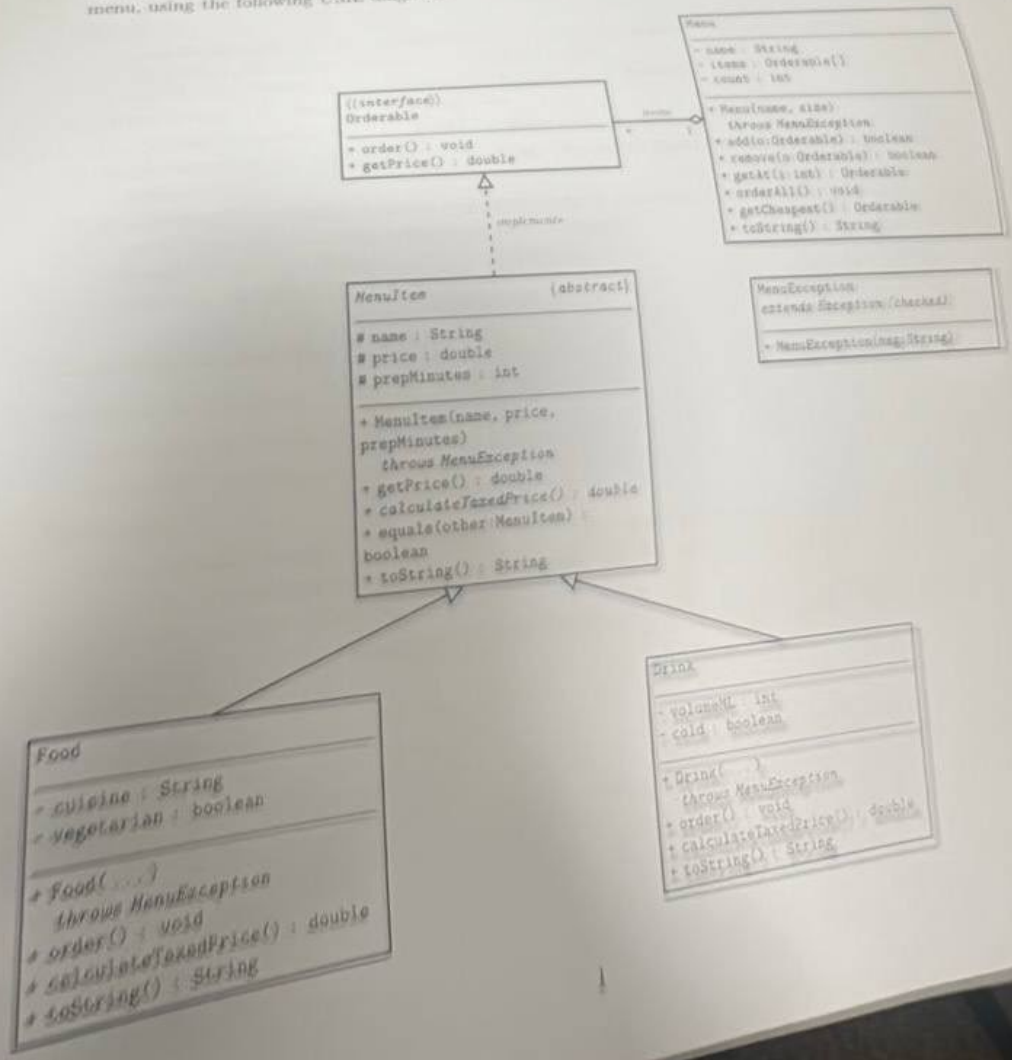
ID: .....

Instructions

- The use of any Java Collection or external library is **not allowed**.
- Open a new workspace before starting in your IDE.
- Write your name and ID at the top of every `.java` file as a comment.
- Submit all `.java` files to LMS. Any file not submitted receives **0**.
- Duration: **1 hour 50 minutes**. Total marks: **10**.

Scenario

We want to write a program that manages orderable items (food and drinks) inside a restaurant menu, using the following UML diagram:



Specification

Interface Discount

Methods (abstract):

- order() : void : orders the item. [0.5 marks]
- getPrice() : double : returns the price of the item in SAR.

Class InvalidOption extends Exception

A checked exception used by MenuItem, Food, Drink, and Sauce. [0.25 marks]

- InvalidOption(String) : constructor that passes message to the superclass constructor.

Abstract Class MenuItem (implements Orderable) [1.5 marks]

Instance Attributes:

- protected name : String
- protected price : double
- protected prepMinutes : int

Methods:

- MenuItem(name, price, prepMinutes) : constructor. Declares throws MenuException. If price < 0, the constructor must explicitly throw: throw new MenuException("Invalid price: " + price);
- getPrice() : double : returns price.
- calculateTaxedPrice() : double : abstract.
- equals(other : MenuItem) : boolean : two menu items are equal if they have the same name.
- toString() : returns a string representation of the item in the following format (< ... > marks an attribute value, everything else is literal):
  <name> (<price> SAR)
  Example: Burger (25.0 SAR)

Class Food extends MenuItem

Instance Attributes: [1.25 marks]

- private cuisine : String
- private vegetarian : boolean

Methods:

- Food(name, price, prepMinutes, cuisine, vegetarian) : constructor. Must call super(name, price, prepMinutes) and declare throws MenuException (propagation – the parent's constructor throws this checked exception).
- order() : void : prints in the following format (< ... > marks an attribute value, everything else is literal):
  Ordering food: <name> [<cuisine>]
  Example: Ordering food: Burger [US]
- calculateTaxedPrice() : double : returns price * 1.15 (15% VAT).
- toString() : returns a string representation of the food in the following format (< ... > marks an attribute value, everything else is literal):
  Food: <name> [<cuisine>] (<price> SAR)
  Example: Food: Burger [US] (25.0 SAR)

Class `Drink` *extends* `MenuItem`

[1.0 marks]

Instance Attributes:

- `private volumeML : int`
- `private cold : boolean`

Methods:

- `Drink(name, price, prepMinutes, volumeML, cold)` : constructor. Must call `super(name, price, prepMinutes)` and **declare** `throws MenuException` (propagation – the parent's constructor throws this checked exception).
- `order()` : `void` : prints in the following format:
  `Pouring drink: <name> (<volumeML>mL)`
  Example: `Pouring drink: Lemon (350mL)`
- `calculateTaxedPrice()` : `double` : returns `price * 1.15`.
- `toString()` : returns a string representation of the drink in the following format (`<...>` marks an attribute value, everything else is literal):
  `Drink: <name> (<volumeML> mL, <price> SAR)`
  Example: `Drink: Lemon (350 mL, 8.0 SAR)`

Note: each child constructor ***must*** *declare* `throws MenuException`. *Children that do not propagate the parent's checked exception will not compile and receive* ***0*** *marks for the constructor.*

Class `Menu`

[3.5 marks]

Instance Attributes:

- `private name : String`
- `private items : Orderable[]`
- `private count : int`

Methods:

- `Menu(name, size)` : constructor. Declares `throws MenuException`.
  If `size <= 0`, the constructor must explicitly throw:
  `throw new MenuException("Invalid size: " + size);`
- `add(o : Orderable) : boolean` : adds `o` to the first available space if there is space and `o` is not already in the menu (use `equals` for comparison). Returns `true` if it succeeds, `false` otherwise.
- `remove(o : Orderable) : boolean` : removes `o` from the menu if it is there by replacing it with the **last** item, then decrements `count`. Returns `true` if it succeeds, `false` otherwise.
- `getAt(i : int) : Orderable` : returns the item at index `i` (index starts from 0), or `null` if `i` is out of range.
- `orderAll() : void` : calls `order()` on every item in the menu.
- `getCheapest() : Orderable` : **recursively** returns the item with the smallest `price` in the menu, or `null` if the menu is empty.
- `toString()` : returns a string representation of the menu in the following format:

```
name Menu (count):
item1
item2
...
```

Note: `getCheapest()` ***must*** *be implemented recursively. An iterative solution will receive* ***0*** *marks.*

Q1 : Implementation

[≈ 7.5 marks total]

Write the interface `Orderable`, the exception class `MenuException`, and the classes `MenuItem`, `Food`, `Drink`, and `Menu` as specified above.

Q2 : `MenuTest` (main method)

[2.0 marks]

Write a class `MenuTest` containing a `main` method that demonstrates the functionality of your work.
All calls that may throw `MenuException` must be wrapped in `try / catch` blocks.
The `main` method should:

1. Inside a single `try / catch (MenuException e)` block:
   (a) First, attempt to create a `Menu` with an **invalid size** (e.g. 0) – the catch should print the message and the program should continue.
   (b) Then create a valid `Menu` named `"Lunch"` with size 4.
   (c) Create the following items and add them to the menu:
   - `Food("Burger", 25.0, 12, "US", false)`
   - `Food("Salad", 18.0, 5, "Med", true)`
   - `Drink("Lemon", 8.0, 2, 350, true)`
   - `Drink("Coffee", 12.0, 3, 250, false)`

   (d) Print the menu.
   (e) Call `orderAll()` on the menu.
   (f) Print the **cheapest** item (using `getCheapest()`).

Hint: place the entire flow inside one outer try/catch since most operations may throw `MenuException`. The first invalid attempt is caught; you may then create the valid menu in a second try/catch or restructure as you wish.

Sample run

```
Invalid size: 0
-----------------------------------------
Lunch Menu (4):
Food: Burger [US] (25.0 SAR)
Food: Salad [Med] (18.0 SAR)
Drink: Lemon (350 mL, 8.0 SAR)
Drink: Coffee (250 mL, 12.0 SAR)
-----------------------------------------
Ordering food: Burger [US]
Ordering food: Salad [Med]
Pouring drink: Lemon (350mL)
Pouring drink: Coffee (250mL)
-----------------------------------------
Cheapest: Drink: Lemon (350 mL, 8.0 SAR)
```